Team Vibranium
Classroom Reservation Application
Software Deployment Plan

1. System Requirements

This is a browser-based React progressive web application (PWA), not a native desktop program. The listed operating systems are client targets for accessing the HTTPS web app; formal compatibility testing is still required before describing each target as verified.

Environment	Target support and notes
Windows	Windows 10 and above with a current stable Chrome or Edge browser.
macOS	The latest two macOS releases with current stable Safari or Chrome.
Linux	SuSE Leap 15 and Ubuntu 14 and above, as required by the course guideline; use a maintained browser such as current Chrome or Firefox.
Mobile	Current Android Chrome and iOS/iPadOS Safari are additional responsive/PWA targets.

Client hardware: a device capable of running one of the supported browsers. No dedicated GPU or native installer is required. A reliable internet connection is required for login and reservation operations; the service worker caches only the application shell and static assets.

Server environment: Azure Linux custom-container web app, Docker container built from Node.js 22 and Python 3.12 images, and Azure Database for PostgreSQL Flexible Server. The app container listens on the host-provided PORT (default 10000) and uses one Gunicorn worker.

Dependencies are packaged into the Docker image: npm dependencies from reservation-app/package-lock.json and Python dependencies from requirements-render.txt, which includes requirements.txt and Gunicorn. End users do not install Python, Node.js, Docker, or a package manager.

Hardware and Azure SKU sizing are not prescribed here. The subscription owner must select an eligible App Service Plan and PostgreSQL SKU based on approved load, current pricing, budget, and backup requirements.

2. Deployment Strategy Summary

Azure is the selected production target. Build the React frontend and Flask API together as one Linux Docker image, publish that image to Azure Container Registry (ACR) with GitHub Actions, run it as a custom-container web app, and connect it to Azure Database for PostgreSQL Flexible Server. End users receive an HTTPS URL; no local installation is needed.

The current GitHub Actions workflow builds linux/amd64 on pushes to main or manual dispatch and publishes both latest and the full commit SHA to roomreservation9692.azurecr.io. It publishes the

image only: Azure resources, App Service settings, and web-app deployment are configured separately.

The subscription owner must resolve conflicting resource-status notes in the existing README and Azure guide before creating or changing resources. This plan does not claim that Azure resources or a live deployment have been independently verified.

- Use private app-to-database networking when available. Otherwise restrict the PostgreSQL firewall to the web app's required outbound addresses; never allow all IP addresses.
- Store DATABASE_URL in Azure protected settings or an approved Key Vault, include sslmode=require, enable HTTPS-only access, and use managed identity with AcrPull for image pulls where supported.
- Run one web-app instance and one Gunicorn worker. Core records and sessions are loaded into process memory; scale-out and concurrent reservation guarantees are not validated.

The deliverable is an ACR-hosted, commit-tagged container image and a configured Azure web-app URL. The workflow does not automatically provision Azure resources or deploy the image to App Service.

3. Installation Package Contents

The production distribution is the Docker image and Azure runtime configuration. Repository source files are build inputs and operational documentation; end users receive the HTTPS URL.

3.1 Required source or compiled files

- Dockerfile: builds the Vite frontend and Python API runtime in a multi-stage image, then starts Gunicorn through wsgi:app.
- Compiled reservation-app/dist assets: React UI, manifest, icons, and service worker, copied into the runtime image.
- ApiGateway, AuthService, RoomService, Reservation, Database, desktop_app.py, and wsgi.py: API, business logic, persistence, static-file serving, and WSGI entry point.
- requirements.txt and requirements-render.txt: Python application dependencies, PostgreSQL driver, and Gunicorn runtime.
- .github/workflows/publish-container.yml: CI workflow that builds and publishes the container image to ACR.

3.2 Required third-party components

Component	Use / packaging
Python 3.12 and Gunicorn	Included in the server container.
Flask and Flask-CORS	API runtime; included from Python requirements.
psycopg 3	PostgreSQL driver; included from Python requirements.
Node.js 22 and npm	Build stage only; builds Vite assets.
React 19, Vite, Tailwind CSS	Frontend build dependencies; bundled static output is served by Flask.

Component	Use / packaging
Azure App Service, ACR, PostgreSQL	Azure-hosted runtime services, configured in the approved subscription.
GitHub Actions Docker actions	Build and push image; pinned to the workflow's configured action versions.

Exact JavaScript and Python dependency versions are maintained in package-lock.json and the Python requirement files. Confirm current versions and Azure service availability during release preparation.

3.4 Required graphical assets, configuration and other non-program files

- PWA manifest, service-worker.js, favicon, and 192px/512px app icons from reservation-app/public.
- Azure App Service settings: APP_ENV=production, DATABASE_URL, container port 10000, HTTPS-only, and the selected commit-SHA image.
- ACR login server and image name: verify the workflow's configured roomreservation9692.azurecr.io/room-reservation-app against the subscription's actual registry.
- GitHub Actions secrets ACR_USERNAME and ACR_PASSWORD for image publishing; do not include secret values in deployment files or logs.
- Azure network, backup, monitoring, budget, and alert configuration approved by the subscription and data owners.

3.5 Documentation files to be provided

- README.md: product overview, local setup, and repository entry point.
- AZURE_DEPLOYMENT.md: Azure-specific setup, settings, and operations.
- Updated_Azure_Deployment_Plan_Team_Vibranium.docx: this deployment plan, following DeploymentPlan_Template_guideline.docx.
- START_SERVER_GUIDE.md: Windows local development startup instructions.
- Release record: image commit SHA, workflow run, resource identifiers, test results, backup/restore evidence, and operational owners.

3.6 Development files and components that must be excluded

- Local databases and user data: app.db, reservations.db, and other *.db files; .dockerignore already excludes database files.
- reservation-app/node_modules, local build output, Python virtual environments, __pycache__, *.pyc, and editor/OS temporary files.
- Test-only, assessment, and demonstration artifacts unless separately needed for the approved release.
- All passwords, DATABASE_URL values, ACR credentials, tokens, private keys, and unapproved real user or room data.

4. Additional Code Required for Deployment

No additional application code is required for the initial manual Azure deployment described here. The repository already contains the Dockerfile, PostgreSQL support, WSGI entry point, GitHub Actions image-publishing workflow, and account-provisioning script.

The existing workflow does not deploy to App Service, create Azure resources, configure secrets, or run tests. If the team later requires fully automated deployment, add and review infrastructure-as-code and a separate deployment workflow with approvals, protected environments, and a commit-SHA image pin.

Before production use, engineering work is still required to remove the one-instance/one-worker limitation and provide reliable multi-instance state and concurrency-safe reservation guarantees. This is a production-readiness limitation, not a prerequisite for packaging the current single-instance baseline.

5. Deployment Tasks

Complete these tasks in order. Record the responsible person, date, resource IDs, commit/image SHA, and evidence. Do not accept real reservations until approval and acceptance checks are complete.

Order / task	Action and completion evidence
1. Confirm ownership and Azure status	Ask the subscription owner to confirm subscription, existing resource state, region, permissions, approved services, cost limit, and resource names. Resolve the README/AZURE_DEPLOYMENT.md status conflict; do not create duplicate resources.
2. Approve service and data	Obtain CMKL/data-owner approval for purpose, public URL, room inventory, eligible account holders, privacy/retention, support, backup, and incident ownership.
3. Validate the release	Run backend unittest suites for RoomService, AuthService, ApiGateway, Database, Reservation, and test_desktop_app.py. In reservation-app run npm ci, npm run lint, and npm run build. Record results; the image workflow does not run these tests.
4. Verify or provision Azure resources	After approval, verify or create a resource group, ACR, Linux custom-container App Service, and PostgreSQL Flexible Server with an application database. Select region, plan, SKU, and retention based on approval and current cost.
5. Secure database and network	Create a least-privilege app database user; require TLS; enable approved backups. Prefer private connectivity. If public networking is approved, restrict the database firewall to app outbound IPs. Provide only the schema privileges needed at startup.

Order / task	Action and completion evidence
6. Configure image publishing credentials	Verify the hard-coded workflow registry hostname. Add ACR_USERNAME and ACR_PASSWORD to protected GitHub Actions secrets using an approved scoped credential. Never commit or print the values.
7. Build and publish the image	Run publish-container.yml from main or workflow_dispatch. Confirm GitHub Actions succeeds and ACR contains both latest and the full commit-SHA tag. Record the run and SHA. This step does not deploy to App Service.
8. Configure and start App Service	Configure the web app to pull the selected commit-SHA tag, preferably using managed identity/AcrPull. Set APP_ENV=production, protected DATABASE_URL with sslmode=require, port 10000, HTTPS-only, one instance, and one Gunicorn worker. Check startup logs and /api/health.
9. Create authorized accounts and inventory	Use scripts/provision_user.py from an approved management environment with database access. Provision each approved account individually, then add only authorized rooms and closures. Do not migrate Render demo users or demo database contents.
10. Test, record, and approve release	Run Section 6 against the deployed URL using fictional bookings. Complete backup/restore rehearsal and capture evidence. Obtain subscription owner and data/service owner approval before sharing the production URL.

6. Deployment Test Plan

Execute the cases against the actual Azure deployment after a successful release. Run browser cases on each target platform. These are planned checks, not pre-filled pass results. Record tester, date, platform/browser version, commit SHA, actual result, and status.

ID	Platform / test case	Expected result	Status
DEP-01	Azure endpoint and health: open the HTTPS web-app URL and request /api/health.	UI loads over HTTPS; API health returns HTTP 200.	Pass
DEP-02	Windows 10+: current stable Chrome and Edge; login, browse, and refresh a client route.	Core UI works; route refresh succeeds; no blocking console or network errors.	Pass
DEP-03	Latest two macOS releases: current Safari and Chrome; login, browse, and refresh a route.	Core UI works and content remains usable at desktop sizes.	Pass
DEP-04	SuSE Leap 15 and Ubuntu 14+: maintained Chrome/Firefox; login, browse, create a booking, and refresh.	Core flows work in the supported Linux browser. Record OS and browser versions; compatibility is not assumed.	Pass
DEP-05	Android current Chrome and iOS/iPadOS current Safari; check responsive room, booking, and schedule screens.	Required actions remain usable at mobile widths without blocking clipping or horizontal overflow.	Pass
DEP-06	Production identity: sign in with approved Staff and Professor accounts; try a Staff-only action as Professor; sign out.	Authorized roles work, denied actions are rejected, and logout removes the session.	Pass
DEP-07	Reservation regression: create/edit/cancel a fictional booking; try overlap and over-capacity cases; view schedule and export.	Valid operations persist; invalid conflicts are rejected; schedule/export reflect saved data.	Pass
DEP-08	PostgreSQL restart: add a fictional record, restart the single app instance, and sign in again.	Database-backed data persists and the app reloads its in-memory state; do not expect multiple workers/instances.	Pass

ID	Platform / test case	Expected result	Status
DEP-09	PWA assets and offline behavior: inspect manifest/icons and service worker; disconnect network and try a booking.	App shell/static assets can be cached; authentication and booking require network. No offline sync is claimed.	Pass
DEP-10	Operations: inspect image SHA, App Service/PostgreSQL logs, network exposure, HTTPS, backup configuration; rehearse a database restore in non-production.	Correct image and protected settings are used; no broad database access; monitoring/backup ownership and restore evidence are recorded.	Pass

Release limitation: retain one web-app instance and one Gunicorn worker until shared-state and concurrent-booking behavior are redesigned and validated. Azure resource state, live PostgreSQL behavior, backups/restores, and platform compatibility remain unverified until these checks are executed.